

Self-Managed Kafka on AKS → Databricks Streaming Pipeline

A real multi-broker Apache Kafka cluster (self-managed, not Event Hubs) on Azure Kubernetes Service, exposed securely over the internet with TLS + SCRAM, streaming into Azure Databricks via Structured Streaming — with credentials in Key Vault.

24 June 2026

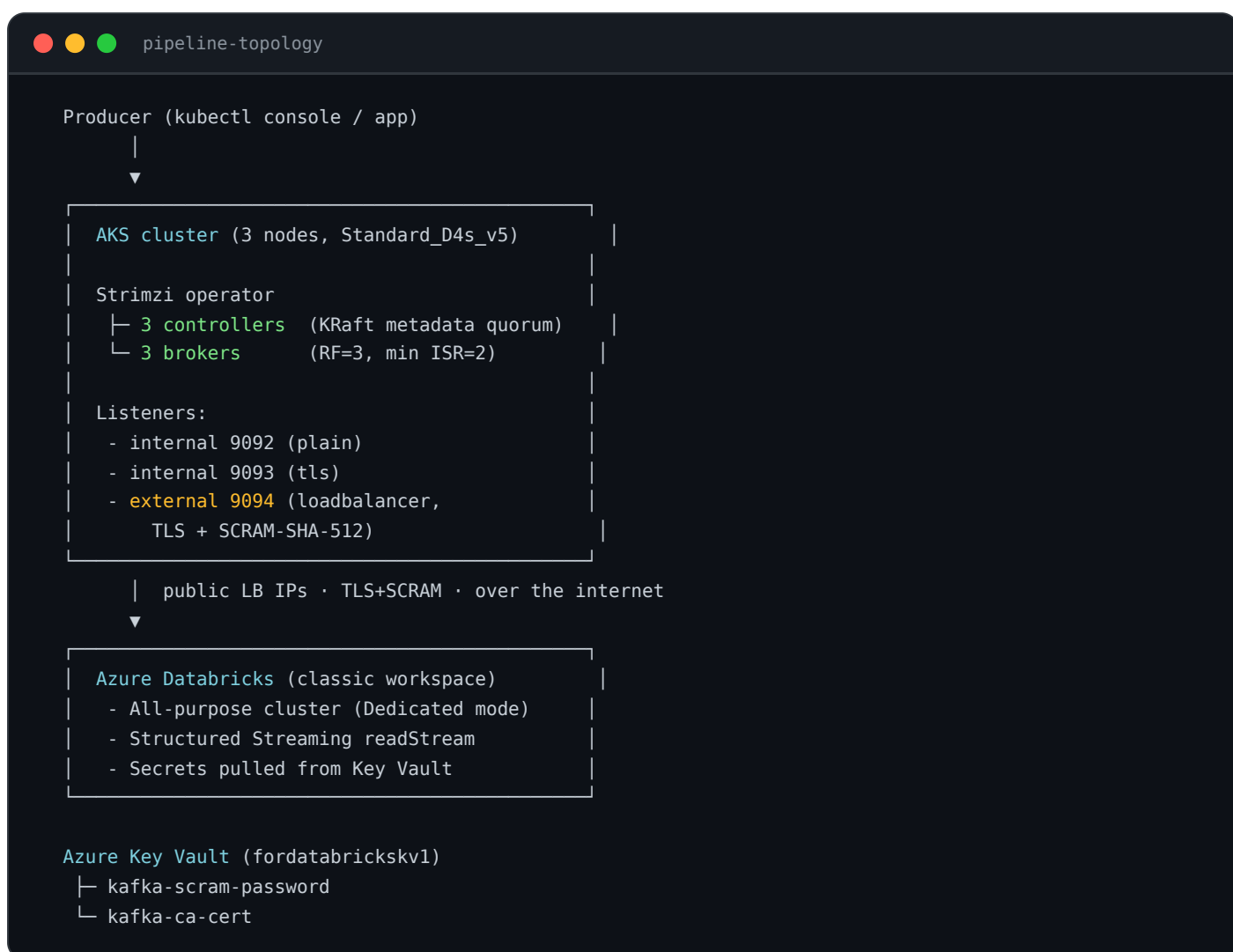
Strimzi · KRaft

3 controllers + 3 brokers

TLS + SCRAM-SHA-512

Key Vault-backed secrets

Final Architecture



Key Decisions & Why

Self-managed Kafka, not Event Hubs. Event Hubs hides the broker/controller topology. To actually run "3

Strimzi operator. The standard way to run Kafka on Kubernetes — declarative KafkaNodePool + Kafka resources, handles certs, listeners, rolling upgrades.

controllers + 3 brokers" you must self-manage → Kafka on AKS via Strimzi.

KRaft mode (no ZooKeeper). ZooKeeper is deprecated/removed in current Strimzi. Dedicated controller nodes hold cluster metadata.

TLS + SCRAM-SHA-512, not mTLS.

Username/password over TLS is far easier to wire into Databricks than client-cert keystores — equally secure here.

Key Vault-backed secret scope. Keeps password + CA cert out of the notebook; pulled at runtime via `dbutils.secrets.get()`, redacted in output.

Path X before Path Y. Proved the cluster worked internally before adding the external listener — isolating "cluster healthy?" from "network/TLS working?" so any failure had a single cause.

loadbalancer listener, not nodeport. Azure provisions proper public IPs; advertised addresses resolve correctly. nodeport is fragile when nodes recycle.

Dedicated-mode Databricks cluster. Standard (shared) mode restricts the filesystem/Spark configs the Kafka SSL setup needs. Dedicated avoids a class of silent failures.

1 Build the AKS Cluster

- ▶ Resource group `rg-kafka-lab` in `centralindia`; 3 nodes `Standard_D4s_v5`, Kubernetes `1.34.5`, `--tier free`.

⚠ **Gotcha — LTS-only version.** Kubernetes 1.30.x was LTS-only (needs Premium tier). Used a `KubernetesOfficial`-tagged version instead. Always check `az aks get-versions --location <region> -o table`.

⚠ **Gotcha — vCPU quota = 0.** DSv5 family had no quota → requested an increase (PAYG, approved fast). A tier upgrade does not grant quota; quota is a separate request.

2 Install the Strimzi Operator

```
kubectl create namespace kafka
kubectl create -f 'https://strimzi.io/install/latest?namespace=kafka' -n kafka
kubectl get crd | grep strimzi # verify CRDs registered
```

3 Deploy Kafka — Path X (internal only)

- ▶ Two KafkaNodePools: `controller` (3 replicas) and `broker` (3 replicas).
- ▶ One Kafka resource: KRaft enabled, internal listeners 9092/9093, RF=3 / min ISR=2.
- ▶ Kafka `4.2.0`, `metadataVersion 4.2-IV1`.
- ▶ Waited for `kubectl get kafka my-cluster -n kafka → READY True`.

⚠ **Gotcha — version pairing.** A mismatched `metadataVersion` crash-loops the brokers. The Kafka version and `metadataVersion` must match the operator's supported pair.

4 Verify Internally — Path X Proof

- ✓ Ran a console consumer + producer as throwaway pods inside the cluster. Watched messages flow producer → broker → consumer. **Cluster proven before exposing anything externally.**

5 Add the External Listener — Path Y

- ▶ Third listener: `external`, port 9094, type `loadbalancer`, `tls: true`, `authentication: scram-sha-512`.
- ▶ Rolling broker restart (one at a time — cluster stayed up).
- ▶ Azure provisioned LoadBalancer services: one bootstrap + one per broker, each with a public IP.
- ▶ External bootstrap: `<bootstrap-LB-IP>:9094`.

6 Create the Kafka User (SCRAM)

- ▶ KafkaUser `databricks-user` with `authentication: scram-sha-512`.

⚠ **Gotcha — ACLs without an authorizer.** An initial `authorization: simple` block with ACLs was rejected ("Simple authorization ACL rules are configured but not supported in the Kafka cluster configuration") because the cluster had no authorizer enabled. Fix: removed the ACL block — SCRAM auth alone suffices for a lab.

7 Extract the Two Secrets

```
kubectl get secret my-cluster-cluster-ca-cert -n kafka \
-o jsonpath='{.data.ca\.crt}' | base64 -d > ca.crt
kubectl get secret databricks-user -n kafka \
-o jsonpath='{.data.password}' | base64 -d > db-password.txt
```

8 Store Secrets in Key Vault

- ▶ Vault `fordatabricksskv1` (RBAC model). Secrets: `kafka-scram-password`, `kafka-ca-cert`.

⚠ **Gotcha — write role.** Writing secrets needs the **Key Vault Secrets Officer** role on your own identity. "Secrets User" is read-only.

9 Grant Databricks Read Access

- ▶ **Key Vault Secrets User** role granted to the global `AzureDatabricks` app — App ID `2ff814a6-3304-4ab8-85cb-cd0e6f879c1d`.

⚠ **Note — shared principal.** This is a tenant-shared identity: all Databricks workspaces in the tenant gain read access to this vault. Fine for a personal lab; isolate with a dedicated vault in production. Secret scopes *cannot* use the Access Connector — they are hardwired to this principal.

10 Create the Key Vault-Backed Secret Scope

- ▶ URL: `https://<workspace-url>#secrets/createScope` (capital S — case-sensitive).
- ▶ Scope `kafka-secrets`, DNS Name = Vault URI, Resource ID = vault resource ID.

⚠ **Gotcha — scope creation role.** Requires Contributor or Owner on the vault, even though reads use RBAC.

11 Verify Secrets Read Back (Isolation Step)

```
print(dbutils.secrets.get(scope="kafka-secrets", key="kafka-scram-password")) # [REDACTED]
cert = dbutils.secrets.get(scope="kafka-secrets", key="kafka-ca-cert")
print(cert.strip().startswith("-----BEGIN CERTIFICATE-----")) # True
print(cert.strip().endswith("-----END CERTIFICATE-----"))      # True
```

- ✓ Confirmed both secrets readable and the PEM round-tripped intact **before** touching Kafka — so any later failure is purely network/TLS/auth, not secrets.

12 Run the Stream

```
KAFKA_BOOTSTRAP = "<bootstrap-LB-IP>:9094"
KAFKA_USER = "databricks-user"
KAFKA_PASSWORD = dbutils.secrets.get(scope="kafka-secrets", key="kafka-scram-password")
CA_CERT = dbutils.secrets.get(scope="kafka-secrets", key="kafka-ca-cert")

kafka_options = {
    "kafka.bootstrap.servers": KAFKA_BOOTSTRAP,
    "subscribe": "my-topic",
    "kafka.security.protocol": "SASL_SSL",
    "kafka.sasl.mechanism": "SCRAM-SHA-512",
    "kafka.sasl.jaas.config": (
        "kafkashaded.org.apache.kafka.common.security.scram.ScramLoginModule '"
        f'required username="{KAFKA_USER}" password="{KAFKA_PASSWORD}";'
```

```

    ),
    "kafka.ssl.truststore.type": "PEM",
    "kafka.ssl.truststore.certificates": CA_CERT,
    "startingOffsets": "earliest",
    "failOnDataLoss": "false",
}

from pyspark.sql.functions import col
df = spark.readStream.format("kafka").options(**kafka_options).load()
parsed = df.select(
    col("key").cast("string"), col("value").cast("string"),
    "topic", "partition", "offset", "timestamp",
)
display(parsed)

```

✓ **Result: streaming table populated. Full pipeline working end to end.**

! **Critical — shaded class name.** The JAAS module must be

`kafkashaded.org.apache.kafka.common.security.scram.ScramLoginModule`. The plain `org.apache.kafka...` name fails on Databricks.

If SSL handshake fails: add `"kafka.ssl.endpoint.identification.algorithm": ""` — Strimzi broker certs won't match a raw IP.

\$ Cost Management (PAYG)

Three independent meters run. The load balancers bill even when AKS is stopped.

AKS worker VMs	3× D4s_v5 — stopped by <code>az aks stop</code> (disks still bill)
LoadBalancer public IPs	Bill independently — only fully stopped by deleting the resource group
Databricks DBUs	Bill while the cluster runs — use auto-terminate

```
az aks stop --resource-group rg-kafka-lab --name aks-kafka-lab # stop VMs
# Terminate the Databricks cluster (or rely on auto-terminate)
az group delete --name rg-kafka-lab # full stop incl. load balancers
```

🔑 Where Credentials Live

- ! Actual secret values are **not** in this document by design.
- SCRAM password → Key Vault secret `kafka-scam-password`
- CA certificate → Key Vault secret `kafka-ca-cert`
- Local copies `db-password.txt`, `ca.crt` — do **not** commit to git.

→ Next Steps

- 1 **Persist the stream into Unity Catalog.** `writeStream` to a UC-managed Delta table (bronze layer) with a checkpoint, instead of the ephemeral `display()`.
- 2 **Build the medallion pipeline.** Bronze → Silver → Gold transformations.
- 3 **Rebuild as a Lakeflow Declarative Pipeline (DLT).** For comparison against raw Structured Streaming.
- 4 **(Optional) Enable cluster-level authorization.** Turn on a Kafka authorizer and re-add scoped ACLs to the `KafkaUser`.